Journal of King Saud University Computer and Information Sciences (2025) 37:86
https://doi.org/10.1007/s44443-025-00059-6

ORIGINAL PAPER

BGNER: A boundary guidance framework for enhanced named entity recognition

Yu He[1,2,3] · Li Sun[1,2,3] · Qianyu Yue[4] · Haitao Wu[1,2,3] · Xiaojuan Wang[1,2,3]

Received: 9 April 2025 / Accepted: 12 May 2025 / Published online: 2 June 2025

Abstract

Named Entity Recognition (NER) is a fundamental task in natural language processing that involves identifying and classifying text segments corresponding to real-world entities. Span-based NER methods, which treat each potential text segment as a candidate entity, have produced strong results. However, these approaches often fail to fully leverage boundary information, as they classify spans independently without explicitly incorporating the context immediately outside the span boundaries. This limitation can result in suboptimal recognition, particularly for nested or ambiguous entities. To overcome this drawback, we introduce BGNER, a method that employs a joint representation of entities and their boundaries. In this approach, each candidate span is enriched not only with an entity type label but also with a boundary label that captures its relationship with surrounding spans. We design a novel neural architecture to exploit this representation: a boundary-aware global pointer module efficiently scores spans by incorporating boundary context, and an entity-guided 3D convolutional neural network captures local patterns between spans and their boundaries. Importantly, predicted boundary spans serve as contextual cues during the decoding stage to validate entity predictions. We evaluate BGNER on six diverse NER datasets spanning both English and Chinese, as well as flat and nested entity scenarios. The experimental results show that our model achieves state-of-the-art performance across all datasets. These findings demonstrate that enhancing span representations with boundary information leads to more accurate and robust entity recognition in various NER settings.

Keywords Natural language processing · Named entity recognition · Boundary guidance · Deep neural networks

1 Introduction

Named Entity Recognition (NER) plays a significant role in natural language processing, serving as a foundation for tasks like information extraction and question answering (Lample et al. 2016; Huang et al. 2015). Traditional NER approaches treat the problem as a sequence labeling task, assigning each token a tag (e.g., using BIO notation) (Collobert et al. 2011; Das et al. 2025). While sequence labeling methods have been very successful, they face challenges in scenarios where entities are nested or overlapping, since a single token can belong to multiple entity spans (Ma and Hovy 2016; Huang et al. 2015; Sun et al. 2019; Chiu and Nichols 2016). To better handle nested entities, span-based methods have emerged as a prominent alternative. Span-based NER explicitly enumerates text spans (contiguous token subsequences) and classifies each span as an entity of a certain type or as non-entity (Sohrab and Miwa 2018; Ke et al. 2024; Zhang et al. 2017; Chaturvedi et al. 2025). This paradigm naturally accommodates nested entities, as multiple spans can overlap. Span-based models, especially when coupled with pretrained Transformers, have achieved state-of-the-art performance on both flat and nested NER (Liu et al. 2021; Zhang and Wang 2023; Alqahtani et al. 2024).

However, current span-based approaches have a notable limitation: they do not fully exploit the information provided by boundary spans, i.e., the regions immediately outside a candidate entity span (Vaswani et al. 2017; Song et al. 2024; Ashok and Lipton 2023; Conneau et al. 2019). Intuitively, the

✉ Haitao Wu
huanghuai20212180@163.com

1 School of Computer Science and Artificial Intelligence, Huanghuai University, Zhumadian 463000, China

2 Henan International Joint Laboratory of Behavior Optimization Control for Smart Robots, Henan 463000, China

3 Henan Key Laboratory of Smart Lighting, Zhumadian 46300, China

4 School of Intelligent Engineering, Xi'an Jiaotong-Liverpool University, Suzhou 215028, China

context around a span's start and end boundaries can offer valuable cues for determining if the span is a valid entity. For example, certain punctuation or trigger words at an entity boundary may indicate the presence or absence of an entity. Existing span models typically represent a span by features of the tokens inside it (such as its start and end token embeddings, or an aggregation of internal token representations) (Liu et al. 2019; Raffel et al. 2020; Strubell et al. 2020). This means that they ignore what lies just outside the span. As a result, they may confuse an entity with a slightly larger or smaller span, or miss entities that lack strong internal cues but have clear boundary indicators. This problem can be exacerbated by the class imbalance in span classification: there are far more non-entity spans than entity spans, and many non-entity spans differ from true entities only subtly at the boundaries (Ha and Grubert 2023; Chi et al. 2021).

In this work, we introduce BGNER to explicitly integrate boundary context into span classification. We propose an entity-boundary span matrix representation that extends the conventional span matrix. In our formulation, each span is characterized by a tuple (i, j, c, b): start index i, end index j, entity type c, and boundary label b. The boundary label b encodes the span's relationship to its neighboring spans (if any). This leads to a multi-dimensional matrix (tensor) where each entry corresponds to a span and is assigned two labels: an entity type and a boundary category. By enriching the span representation with boundary labels (such as indicating if a span covers the left or right context of an actual entity), we balance the sample space of spans and provide the model with richer supervision. Intuitively, the model learns not only which spans are entities, but also learns to recognize spans that are immediately outside entities. This additional knowledge acts as a guide, helping the model distinguish true entity boundaries from near-misses. As a result, our approach captures the spatial relationships between an entity span and its context in a structured way that traditional methods lack.

Building on this representation, we develop an entity-boundary span neural architecture to predict the joint span-boundary matrix for each input sentence. Our architecture consists of three main components: (1) a Category-Enhanced Encoder based on BERT and BiLSTM that encodes the input sequence along with special tokens representing entity types and boundary types, ensuring that the token representations are enriched with category information; (2) a Boundary-Guided Global Pointer module that efficiently scores all spans by leveraging boundary token embeddings to modulate token representations via Conditional Layer Normalization (CLN), extending the GlobalPointer mechanism with boundary awareness; and (3) an Entity-Guided Three-Dimensional Convolutional Neural Network (3D CNN) module that performs convolution over the span matrix, treating entity type dimensions as separate channels. The pointer module focuses on identifying spans and their entity types (using boundary cues), while the 3D CNN focuses on local patterns of spans and boundary labels (conditioned on entity type). By combining these, our model learns a span-boundary scoring function that yields a filled entity-boundary span tensor for a given sentence.

During inference, we decode entities from the predicted tensor by looking for spans labeled as actual entities (boundary label C for "Center") and then validating them against their boundary predictions. In other words, a span is output as an entity only if it is not only scored as an entity span but also has the appropriate boundary spans around it labeled as start/end (S, E, or SE). This novel decoding step uses the predicted boundary spans as a criterion for entity determination, effectively filtering out false positives that are not supported by consistent boundary evidence. By requiring, for example, that an entity's immediate left neighbor span is labeled as a Start-boundary and the right neighbor as an End-boundary, we greatly improve precision in identifying exact entity boundaries.

We conduct extensive experiments on six NER datasets to evaluate the effectiveness of our approach. Our model, BGNER, achieves state-of-the-art performance across all six benchmarks, outperforming a wide range of baseline models. Our main contributions are as follows:

- We propose a novel multi-labeled span matrix representation that integrates both entity and boundary labels.
- We design a unified model architecture that combines a boundary-aware global pointer with an entity-informed 3D CNN to effectively capture span-boundary interactions.
- We introduce a decoding strategy that leverages boundary predictions to enhance entity extraction accuracy.

2 Related work

2.1 Flat NER

Early work in Named Entity Recognition (NER) approached the task as a sequence labeling problem. One classic methodology is the use of Conditional Random Fields (CRFs) to capture label dependencies (Lafferty et al. 2001; Sutton and McCallum 2006). Before deep learning became prevalent in NER, methods from speech recognition and other sequential tasks were influential-for instance, Hidden Markov Models were widely studied for their application in speech (Rabiner 1989; Parsing 2009), and framewise classification with bidirectional LSTM architectures was demonstrated for phoneme recognition (Graves and Schmidhuber 2005).

With the advent of neural networks, the BiLSTM-CRF model became a de facto baseline. Huang et al. showed that integrating a bidirectional LSTM with a CRF layer could

effectively model long-range dependencies and enforce valid sequence constraints (Huang et al. 2015). Additional efforts extended these ideas to domain-specific applications, such as applying deep neural network architectures to clinical text for NER (Wu et al. 2015). More recently, adversarial training with lattice LSTMs has been introduced to handle domain-specific texts like rail fault descriptions (Su et al. 2022).

Lexicon information has played an important role in flat NER, especially for languages with complex word segmentation issues such as Chinese. Early work incorporating lexicon features using pointer networks demonstrated substantial gains in performance, and subsequent efforts revisited and refined this technique using improved lexicon integration strategies (Guo and Guo 2022). In parallel, Transformer-based models have significantly influenced the field. For example, TENER adapts the Transformer by integrating relative positional encodings, thus capturing global context for both character-level and word-level inputs (Dai et al. 2019). Deep contextualized representations have also been proven effective in extending NER performance in challenging contexts, as illustrated in work on related phenomena like formal thought disorder detection (Sarzynska-Wawer et al. 2021).

Generative pre-training approaches and encoder-decoder frameworks have further enriched flat NER. In particular, denoising sequence-to-sequence models such as BART (Lewis et al. 2019) and conditional Transformer language models (Keskar et al. 2019) have inspired ideas for controlled generation, while research on neural text degeneration has highlighted the challenges of generative methods (Holtzman et al. 2019). Multi-task frameworks combining instruction and chain-of-thought prompting have also been proposed for few-shot NER (Xu and OuYang 2023). Comprehensive surveys provide an overview of these developments, highlighting advances in both methodology and application domains (Jehangir et al. 2023; Pakhale 2023; Yadav and Bethard 2019; Thomas and Sangeetha 2020; Roy 2021; Yang et al. 2024).

2.2 Nested NER

Nested NER, in which entity spans may overlap partially or wholly, poses additional challenges that have motivated various methodological developments. Early approaches included layer-based methods-often employing stacked BiLSTM-CRF networks-to detect nested entities in multiple passes, with inner entities recognized before outer ones (Sohrab and Miwa 2018). Models such as the segment-enhanced span-based method extend this idea by refining span representations within each layer (Li et al. 2021). Pyramid structures have been introduced to overcome the limitation of predefining the maximum nesting depth, offering a more flexible hierarchical approach (Wang et al. 2022).

Alternative formulations treat NER as a parsing or structured prediction problem. Some works reformulate the task as dependency parsing (Yu et al. 2020), while others propose structured span prediction frameworks that account for the full set of possible spans (Zaratiana et al. 2022). Exogenous data augmentation methods have been explored for low-resource complex NER scenarios, blending both external and internal data sources to improve performance (Zhang et al. 2024). Unified generative frameworks have also been proposed that reframe nested NER as one of several complementary subtasks (Yan et al. 2021), and augmented sequence labeling strategies have been advanced to better leverage the structure inherent in the data (Athiwaratkun et al. 2020).

Boundary information is a key challenge in nested NER. Two-stage frameworks that "locate and label" candidate spans have shown promise in enhancing the precision of entity detection (Shen et al. 2021). Complementary to these, methods that directly embed boundary features-either through boundary smoothing techniques (Zhu and Li 2022) or by developing boundary-aware neural models (Zheng et al. 2019)-have advanced the state of the art. Some studies further integrate uncertainty estimations into the recognition process to improve model calibration (Hu et al. 2023). Innovations continue with proposals for grid-tagging networks to capture mention structures (Jia et al. 2024) and classifiers that incorporate multi-feature word-to-word relationship information to refine span predictions (Jiang 2024). Dedicated boundary feature enhancements for nested structures have been specifically targeted in recent work (Song et al. 2024), and alternative methods rethinking discontinuous mentions using pointer networks have been proposed (Fei et al. 2021). Finally, efficient span-based methods leveraging global pointer techniques have been shown to further balance computational cost and accuracy (Su et al. 2022), while pretrained language models continue to push the limits of span-based nested recognition (Liu et al. 2021).

Cross-lingual challenges have also garnered attention, with techniques applied to Japanese NER via transfer learning (Johnson et al. 2019) and innovative multi-view contrastive learning methods proposed for cross-lingual settings (Mo et al. 2024). Additionally, joint entity and relation extraction methods extend the span-based paradigm to capture interactions beyond simple entity boundaries (Du et al. 2024).

2.3 Other approaches and trends

Beyond direct adaptations of flat or nested NER models, the broader research community has explored several trends and alternative formulations. Multi-modal approaches have been proposed that integrate spatially invariant CNNs from computer vision into language tasks, offering insights into robust feature extraction across modalities (Asif et al. 2017). Recent

work on levitated controlled attention introduces novel attention mechanisms to further refine NER outputs (Huang et al. 2025), and new models enhancing span recognition with context-based label knowledge have also been developed (Chen et al. 2024).

There is a growing interest in frameworks that treat NER as a controllable generation problem. In this vein, conditional generative models like CTRL (Keskar et al. 2019) and its object-centric extensions (Didolkar et al. 2025) offer promising yet computationally intensive alternatives. Research on generative models is not confined to traditional text; for example, a generative approach was tailored for geological NER tasks (Qiu et al. 2019), and domain-specific studies have applied large-scale foundation models to autonomous driving scenarios, emphasizing the versatility of NER techniques across application domains (Huang et al. 2023).

Furthermore, investigations into cross-lingual and language-specific adaptations have yielded promising results. In Korean, specialized deep contextualized word representations have been deployed to improve performance (Hong et al. 2018). Cross-lingual transfer approaches have been successfully applied to Japanese NER (Johnson et al. 2019), and recent multi-view contrastive learning methods have achieved impressive gains in cross-lingual settings (Mo et al. 2024). Altogether, these diverse approaches indicate that NER research continues to evolve rapidly, guided by a rich interplay of ideas from structured prediction, generative modeling, and multi-modal learning.

3 Methodology

3.1 Formulation of the joint span-boundary label matrix

Traditional span-based NER methods enumerate all possible text spans and classify each span as either an entity of a certain type or non-entity. Formally, a span can be represented by a triple (i, j, t) indicating the start index i, end index j, and the entity type t of the span (if any). These approaches construct a span matrix of size $L \times L$ (for sentence length L) where each cell corresponds to a candidate span, and the cell's value is the predicted entity type. However, a limitation of such approaches is that they do not explicitly leverage information about boundary spans-the neighboring regions immediately outside a candidate span-which can provide valuable cues for entity recognition. We address this by formulating a joint span-boundary label matrix that enriches the span matrix with boundary information.

In our formulation, each span is represented as a quadruple (i, j, t, b), where i and j are the span's start and end positions, t is the entity type, and b is a boundary label indicating the span's relation to its surrounding context. This effectively creates a multi-dimensional matrix (or tensor) that captures the spatial relationship between an entity span and its adjacent boundaries. Concretely, we allocate a separate $L \times L$ matrix for each entity type, and in each such matrix the entry at (i, j) is filled with a boundary label b. All these per-type matrices together form a tensor of size $N \times L \times L$, where N is the number of entity categories. Each cell in this tensor thus specifies a span (via its i, j coordinates) and associates two kinds of labels with it: an entity category (by virtue of the matrix "layer") and a boundary category (the cell value). The boundary label can take one of the following values:

- **C (Center)**: Indicates that the span itself is an entity. This label marks the center span of an entity, i.e. the span (i, j) is recognized as an actual entity span of some type.
- **S (Start)**: A span labeled S denotes a start-boundary span. This means the span (i, j) corresponds to a context where the end j aligns with the end of an entity, but the start i is one token before the entity's start. In other words, (i, j) covers an entity's right-boundary context.
- **E (End)**: A span labeled E denotes an end-boundary span, i.e. span (i, j) starts at the beginning of an entity and extends one token beyond the entity's end, aligning with the start of the entity.
- **SE (Start-End)**: This label indicates a span that covers both the left and right boundaries around an entity. An SE span (i, j) starts one token before an entity and ends one token after the entity, thus spanning both boundaries.
- **None**: If a span does not correspond to any of the above boundary cases (i.e., it is not adjacent to any entity span of the given type), it is labeled None (left as blank in the matrix). This is the default label for irrelevant or non-entity spans.

All boundary labels are defined in relation to an actual entity span labeled C. It is important to note that without a C in a given entity-type layer, the other boundary labels S, E, SE should not appear in that layer independently. In other words, boundary labels serve to annotate the context around a genuine entity span. Compared to traditional boundary-only encodings such as BIO or BILOU-which mark only "begin," "inside," and "last" positions and thus often confuse near-boundary or non-entity spans. This multi-tiered design not only filters out spurious spans early (via C and S) but also demands consistent boundary evidence (via E and SE),

thereby distinguishing true entities from near-boundary or non-entity spans with far greater accuracy.

Before model training, we initialize the joint span-boundary tensor for each input sentence. We create N matrices of size $L \times L$ (where N is the number of entity types and L is sentence length) and initialize all cells to None. Then, given the ground-truth entity annotations, we populate these matrices with boundary labels as follows. For each annotated entity of type c spanning token positions i through j: (1) We label the span (i, j) in the c-th matrix with C (center). (2) If the span $(i-1, j)$ (one token to the left extending to the entity end) falls within the sentence and is currently unlabeled, we label it S (start-boundary). (3) If the span $(i, j+1)$ (entity start through one token to the right) exists and is unlabeled, we label it E (end-boundary). (4) If the span $(i-1, j+1)$ (one token beyond both ends) exists and is unlabeled, we label it SE (start-end boundary). Furthermore, if a span was already labeled S or E and step (4) applies, we update that span's label to SE to indicate it actually covers both boundaries. This procedure ensures that for every true entity span C, its immediate neighboring spans are appropriately marked with boundary labels, providing a richer context representation around the entity.

For example, suppose we have a sentence containing the entity "New York" (a location) spanning token positions i through j. In the Location matrix (layer) of the tensor, we would mark cell (i, j) with C to denote that "New York" is an entity. Then cell $(i-1, j)$ (if it exists in the sentence) would be labeled S (since it ends at the entity's end j), cell $(i, j+1)$ would be labeled E, and cell $(i-1, j+1)$ would be SE if it exists. If any of those cells were already singly labeled, we adjust them to SE as needed. After processing all entities in the sentence in this manner, we obtain a fully labeled joint span-boundary tensor (essentially an enriched span matrix) that will serve as the supervision target for our model. This joint representation balances the span classification task by injecting boundary context, which we hypothesize leads to more effective learning of span boundaries and entity extents.

3.2 Neural architecture for span-boundary integration

Our proposed model aims to predict the above span-boundary label tensor for each sentence. Figure 1 illustrates the architecture of our span-boundary integrated network, which comprises three main components: (i) a Category-Enhanced Encoder that produces contextual representations of tokens as well as the specially introduced category tokens, (ii) a Boundary-Guided Global Pointer module that leverages boundary category information to help identify entity spans (entity categories and span positions), and (iii) an Entity-Guided 3D CNN module that leverages entity type information to help predict the boundary labels for each span. The encoder feeds its outputs into the two specialized modules in parallel; the two modules focus on complementary aspects (entity span identification vs. boundary classification) and their outputs are finally fused to produce a rich span representation matrix. The fused output is a four-dimensional score tensor from which the joint span-boundary label matrix is decoded. During decoding, the predicted boundary labels guide the extraction of entity spans, ensuring that the model's proposed entities are consistent with their boundary contexts.

3.2.1 Category-enhanced encoder

To enable the model to concurrently capture entity type information and boundary information from the input, we introduce special category tokens into the input sequence and

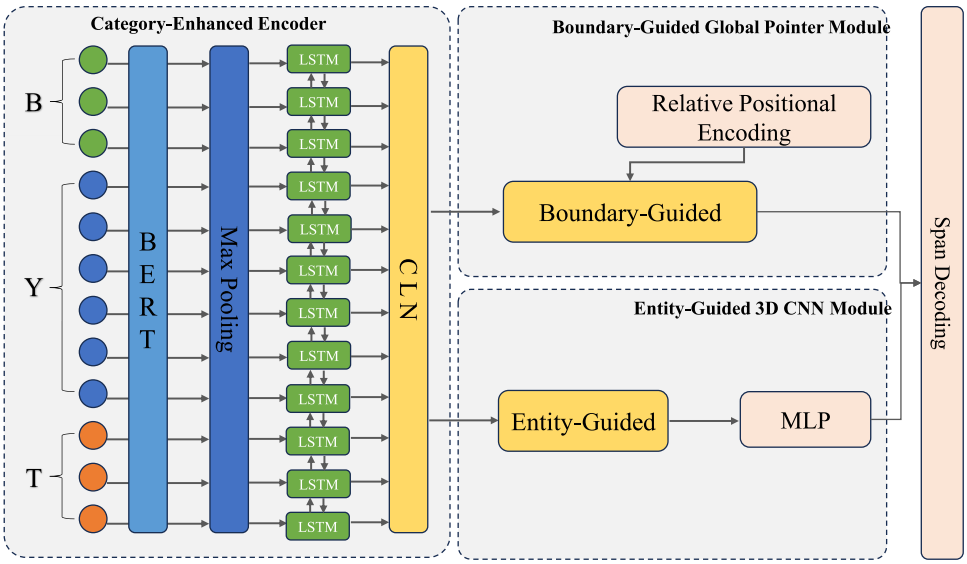

Fig. 1 Overview of the BGNER

process them with a dual encoder. In particular, we use two sets of learnable markers: one set for boundary categories and another set for entity categories. Let M be the number of boundary label categories (for our task $M = 5$, corresponding to C, S, E, SE, None), and let N be the number of distinct entity types. We denote the sequence of boundary category tokens as $B = b_1, b_2, \ldots, b_M$, and the sequence of entity category tokens as $Y = y_1, y_2, \ldots, y_N$. Each b_m and y_n is a learned embedding meant to represent a particular boundary or entity class. Given an input sentence of length L (with token sequence $T = t_1, t_2, \ldots, t_L$), we prepend these category tokens to form an augmented input sequence. Formally, we define:

$$Z = B \parallel Y \parallel T \tag{1}$$

where $\parallel$ denotes concatenation. In other words,

$$Z = b_1, \ldots, b_M, ; y_1, \ldots, y_N, ; t_1, \ldots, t_L \tag{2}$$

This expanded sequence Z is then fed into a pretrained Transformer encoder (BERT) to obtain initial contextualized representations. We utilize BERT due to its proven effectiveness in capturing rich context for NER tasks. Each token (including each special category token and each original word token) is split into subword pieces as needed and processed by BERT. We then apply a pooling operation (max-pooling in our implementation) over the subword pieces of each token to produce a single vector representation for that token. This yields a sequence of contextual embeddings

$$R = r_1, r_2, \ldots, r_{M+N+L} \tag{3}$$

where $r_1, \ldots, r_M$ correspond to the boundary tokens, r_{M+1}, $\ldots, r_{M+N}$ correspond to the entity type tokens, and the remaining $r_{M+N+1}, \ldots, r_{M+N+L}$ correspond to the actual sentence tokens.

Next, we further refine these representations using a Bidirectional LSTM (BiLSTM) layer. The BiLSTM processes the sequence R in both forward and backward directions to capture long-range dependencies beyond the Transformer's subword context. Let $\overrightarrow{h}\,k$ be the hidden state produced by the forward LSTM at position k (reading the sequence from r_1 to r_{M+N+L}), and $\overleftarrow{h}\,k$ be the hidden state of the backward LSTM at the same position k (reading from r_{M+N+L} backwards to r_1). The BiLSTM updates these states as:

$$\overrightarrow{h}_k = \text{LSTM}_{fwd}(r_k, \overrightarrow{h}_{k-1}) \tag{4}$$

$$\overleftarrow{h}_k = \text{LSTM}_{bwd}(r_k, \overleftarrow{h}_{k+1}) \tag{5}$$

for $k = 1, \ldots, M + N + L$. We then concatenate the forward and backward hidden states at each position to obtain the final BiLSTM output

$$h_k = [\overrightarrow{h}_k, \overleftarrow{h}_k,] \tag{6}$$

The sequence of encoder outputs is

$$H = h_1, h_2, \ldots, h_{M+N+L} \in \mathbb{R}^{(M+N+L)\times d} \tag{7}$$

where d is the dimension of the concatenated hidden state. This encoding H contains contextual information that now includes knowledge of the category tokens and the original text. We logically partition H into three parts corresponding to the original segments of the input Z: (a) Boundary category representations $H_B = h_1, \ldots, h_M$, which are the contextual embeddings of the boundary label tokens B; (b) Entity category representations $H_Y = h_{M+1}, \ldots, h_{M+N}$ for the entity type tokens Y; and (c) Token representations $H_T = h_{M+N+1}, \ldots, h_{M+N+L}$ for the actual sentence tokens. These subsets H_B, H_Y, and H_T will serve as inputs to subsequent modules. Intuitively, H_B encodes information about general boundary label semantics in the context of the sentence, H_Y encodes the semantic space of entity types in that context, and H_T encodes the content of the sentence itself, all in the same vector space. By incorporating H_B and H_Y, the encoder is category-enhanced - it has forced the model to represent boundary and entity type information alongside token features, which we expect to ease the learning of interactions between them in later stages.

3.2.2 Boundary-guided global pointer module

After obtaining the encoder representations, the first specialized sub-network aims at leveraging boundary information to guide the identification of entity spans. Prior work has shown that span-based global pointer mechanisms can efficiently enumerate and score spans for NER. Here, we adopt the efficient Global Pointer approach and augment it with boundary-awareness. In essence, our Boundary-Guided Global Pointer module uses the boundary category representations H_B (from the special tokens) to condition the token representations H_T, producing boundary-informed token features that are then used to score spans.

Boundary-aware token representation via CLN We generate a new representation U that integrates each token's features with a specific boundary category context. For each boundary category $m \in 1, \ldots, M$ and each token position $k \in 1, \ldots, L$, we construct an embedding $u_{m,k}$ that represents "token k under boundary category m." To achieve this, we apply Conditional Layer Normalization (CLN), a

mechanism that uses one conditioning vector (here, a boundary category embedding) to modulate the normalization of another vector (a token embedding). Let $h_k^{(T)}$ be the k-th token's representation from H_T, and $h^{(B)}m$ be the m-th boundary token's representation from H_B. We define $u_{m,k} = \text{CLN}(h_k^{(T)} \mid h_m^{(B)})$, which can be expanded as:

$$u_{m,k} = \gamma_{m,k} \odot \text{Norm}(h_k^{(T)}) + \beta_{m,k} \tag{8}$$

where Norm$(\cdot)$ denotes normalization (e.g., instance normalization) applied to the token representation $h^{(T)}k$, and the scale $\gamma_{m,k}$ and bias $\beta_{m,k}$ are conditioned on the boundary representation $h^{(B)}m$. Specifically,

$$\gamma_{m,k} = W_\gamma h_m^{(B)} + b_\gamma \tag{9}$$

$$\beta_{m,k} = W_\beta, h_m^{(B)} + b_\beta \tag{10}$$

are learned linear projections of the boundary vector $h_m^{(B)}$ (with $W\gamma$, $W_\beta \in \mathbb{R}^{d\times d}$ and bias vectors $b_\gamma, b_\beta \in \mathbb{R}^d$). Intuitively, the CLN operation scales and shifts the normalized token embedding $h_k^{(T)}$ according to the boundary category context. The outcome um, k can be interpreted as the token t_k embedding imbued with "what if boundary category m is active" information. Collecting all such $u_{m,k}$, we form a tensor $U \in \mathbb{R}^{M\times L\times d}$ of boundary-aware token representations (sometimes we refer to this as V in equations for clarity). Each slice $U[m, :, :]$ is a length-L sequence of tokens conditioned on boundary type m, and each slice $U[:, k, :]$ is the set of M different boundary-conditioned variants of token k.

Span scoring with efficient global pointer: Using the enriched representation U, the module next produces scores for each possible span (i, j) (start at i, end at j) and for each entity type. We employ the efficient Global Pointer mechanism, which uses a bilinear form of attention to score span boundaries, combined with classification logits for entity types. The process can be described in two parts: (a) span extraction - identifying a span regardless of type, and (b) span classification - predicting the entity type of that span. These are done in a unified scoring formula.

First, we compute two sets of vectors from U that will represent span start and span end. Specifically, we apply two linear transformations to U: one to produce a query vector $q_{m,k}$ and one to produce a key vector $k_{m,k}$, for each position k and boundary m. Formally:

$$q_{m,k} = W_q u_{m,k} + b_q \tag{11}$$

$$k_{m,k} = W_k u_{m,k} + b_k \tag{12}$$

where $W_q, W_k \in \mathbb{R}^{d\times d}$ and $b_q, b_k \in \mathbb{R}^d$ are trainable parameters. The vectors $q_{m,k}$ and $k_{m,k}$ live in the same d-dimensional space. Intuitively, $q_{m,k}$ can be viewed as an embedding representing a span starting at token k (with boundary context m), and $k_{m,k}$ represents a span ending at token k (with boundary context m). Given a candidate span from position i to j in the sentence, and considering a particular boundary category m, we will use $q_{m,i}$ and $k_{m,j}$ to characterize that span's boundaries.

In parallel, we also want to predict the entity type of each span. For this, we take the boundary-conditioned token representations at the span's start and end positions ($u_{m,i}$ and $u_{m,j}$) and concatenate them. This gives a feature vector $[u_{m,i}; u_{m,j}]$ of length $2d$ which represents the span's content (with boundary m) at its edges. We associate with each entity type β a learned weight vector $\omega_\beta \in \mathbb{R}^{2d}$ that will act as a classifier for that span feature. The inner product $\omega_\beta^\top [u_{m,i}; u_{m,j}]$ produces a score indicating how likely the span (i, j) is of entity type β.

Finally, we combine the boundary-based span extraction score and the entity classification score. The overall score for a span starting at i and ending at j (inclusive) with entity type β and considering boundary context m is given by:

$$s_{m,\beta}(i, j) = q_{m,i}^\top k_{m,j} + \omega_\beta^\top [u_{m,i}; u_{m,j}] \tag{13}$$

The first term $q_{m,i}^\top k_{m,j}$ can be seen as a similarity score between the span's start and end representations, which is high when tokens i and j have a configuration indicative of a valid entity span (this term essentially scores the existence of a span regardless of type). The second term $\omega_\beta^\top [u_{m,i}; u_{m,j}]$ injects the entity type-specific score, evaluating how well the span's edges match the profile of type β. In effect, the model is assigning a confidence that "span (i, j) is an entity of type β" while also accounting for a particular boundary condition m.

To further improve span scoring, we incorporate Relative Position Encoding (RPE). RPE explicitly encodes the distance between i and j (span length) into the dot-product term to bias the model with positional context. We can implement this by multiplying one of the boundary representations by a rotation or phase $R^{(j-i)}$ dependent on the length of the span. With RPE, the span score becomes:

$$s_{m,\beta}(i, j) = q_{m,i}^\top \mathbf{R}^{(j-i)} k_{m,j} + \omega_\beta^\top [u_{m,i}; u_{m,j}] \tag{14}$$

where $\mathbf{R}^{(j-i)}$ is a length-dependent diagonal matrix (or equivalently a function that injects a sinusoidal phase for the relative position $j - i$). The second term remains the same. If two spans have the same start and end tokens but different distances, their scores will differ according to $R^{(j-i)}$,

enabling the model to learn, for example, priors that very long spans or very short spans might be less likely for certain entity types.

By evaluating $s_{m,\beta}(i, j)$ for all possible $i \leq j$, all boundary labels m, and all entity types β, the Boundary-Guided Pointer module produces a four-dimensional score tensor. We denote this output tensor as $P \in \mathbb{R}^{N \times L \times L \times M}$. Here, the first dimension indexes the entity type β (since effectively the score incorporates β), and the last dimension indexes the boundary label m. In other words, for each entity type c and each candidate span (i, j), $P[c, i, j, :]$ is a length-M vector containing scores for that span being labeled with each boundary category (C, S, E, SE, None) according to the pointer module. Equivalently, one can think of P as N slices (one per entity type), each slice being an $L \times L \times M$ score cube for spans and boundary labels. These scores will later be combined with those from the CNN module and passed through a softmax to yield probability distributions.

3.2.3 Entity-guided 3D CNN module

While convolutional neural networks (CNNs) have been found effective in modeling structural relationships in span-based NER grids, we further adopt them here due to their structural advantages over attention-based mechanisms for modeling fine-grained local dependencies. CNNs provide a strong spatial inductive bias that makes them particularly suitable for capturing the localized interactions between span boundaries, such as start and end positions of entity spans. Unlike Transformer-based attention mechanisms, which model all token pairs equally and require learned positional encodings to retain spatial order, CNNs inherently preserve and exploit spatial locality through their kernel structure. This is especially valuable in span-boundary prediction tasks, where interactions between closely situated tokens often provide critical labeling cues. Additionally, our use of 3D CNNs allows the model to effectively integrate multi-dimensional span information (e.g., span width, location, and entity context) in a parameter-efficient and computation-friendly manner, without the overhead of global self-attention. Empirically, we also observe that CNN-based modules outperform their attention-based counterparts in capturing local entity boundary patterns across multiple datasets, justifying our design choice.

While the pointer module focuses on capturing entity type information for spans, we also want to capture the local spatial relationships between spans and their neighboring boundaries. Convolutional neural networks (CNNs) have been found effective in modeling structural relationships in span-based NER grids. Motivated by this, our second module employs a CNN-based approach that is guided by entity category information to predict boundary labels for spans. We term this the Entity-Guided 3D CNN module because it uses the entity type context to inform a three-dimensional convolution over the span matrix.

Entity-aware span representation As a first step, we fuse the entity category representations (from the encoder) with the token representations to create an initial span-wise feature map. We begin with the token representation sequence $H_T = h_1^{(T)}, \ldots, h_L^{(T)}$ from the encoder. To ensure these token features are tuned for span-level processing, we apply a self-conditional normalization, similar in spirit to the CLN above but conditioning on itself (this can be viewed as a form of layer normalization on H_T). Let S denote the initial span feature matrix derived from H_T. For simplicity, one can imagine S as initially an $L \times L$ matrix where each entry (i, j) is intended to represent features of the span (i, j). In practice, constructing S explicitly for all spans is memory-intensive; instead, we implicitly handle it via convolution operations. In the paper, this step is described as:

$$S = \mathrm{CLN}(H_T \mid H_T) \tag{15}$$

which indicates we normalize the token representations with a mechanism that could involve their own statistics (effectively preparing them for combination). Then, following (Jehangir et al. 2023), we enrich these span features with two types of additional information: relative position and region indicators. The relative position feature S_d encodes information about the length or distance of spans (for instance, it could encode $j - i$ or positional buckets indicating how far apart i and j are). The region feature S_t encodes the segment identity of the span (e.g., whether a token is at the span's boundary or inside). We concatenate these features with S and use a multi-layer perceptron to blend them:

$$S = \mathrm{MLP}\left([S; S_d; S_t]\right) \tag{16}$$

After this step, S is an $L \times L \times d'$ tensor representing span candidates, now enriched with positional and region information (where d' is the updated feature dimension). Each cell (i, j) in S can be interpreted as a feature vector describing the span from i to j in context (but not yet conditioned on any particular entity type).

Next, we incorporate the entity category context. We use the entity token representations

$$H_Y = h_1^{(Y)}, \ldots, h_N^{(Y)} \tag{17}$$

from the encoder (where each $h_n^{(Y)}$ corresponds to an entity type n). For each entity type n, we want to condition the span features S on that type. We again use a conditional layer normalization approach: we treat the span feature matrix S as the

primary input and the entity type embedding $h_n^{(Y)}$ as a conditioning vector. Essentially, we create N different versions of the span feature matrix, each "colored" by a different entity type embedding. Denote the resulting entity-informed span representation as C. Formally, for each entity type n:

$$C_n = \mathrm{CLN}\left(S \mid h_n^{(Y)}\right) \tag{18}$$

and we stack these so that $C = C_1, C_2, \ldots, C_N \in \mathbb{R}^{N \times L \times L \times d'}$. Each C_n is an $L \times L$ matrix of span features conditioned on entity type n. In this way, C is analogous to the structure of our joint span-boundary matrix: it has a slice for each entity type. Within each slice (entity type n), a cell (i, j) contains a feature vector encoding the span (i, j) assuming entity type n is the type of that span. The conditioning allows the model to potentially encode how the presence of a particular entity type might influence the surrounding context patterns (for instance, certain entity types might have distinctive boundary signal patterns).

3D convolution over span-boundary grid: Now that we have an entity-informed span representation C, we apply three-dimensional convolutional neural networks to capture local patterns in this 3D volume. A 3D convolution in this context slides a 3D filter across the (i, j) span dimensions and the entity-type dimension, capturing interactions among neighboring spans and potentially between neighboring entity type hypotheses. In practice, we keep the entity type dimension separate (since different slices don't interact directly), and perform 3D convolutions mainly over the two spatial dimensions (span start i and span end j) and the feature depth. Each 3D convolutional layer considers a small neighborhood of spans (e.g., spans that start or end within a certain offset of each other) and computes higher-level features. We employ multiple such convolutional layers with different dilation rates l to capture interactions at various scales. Dilation $l = 1$ considers immediately adjacent spans, $l = 2$ skips one span in between, $l = 3$ skips two, etc., enabling coverage of wider context without using many layers. For a given dilation rate l, the output of the 3D Conv layer is:

$$Q^{(l)} = \sigma\left(\mathrm{3DConv}_{(l)}(C)\right) \tag{19}$$

where $\mathrm{3DConv}_{(l)}$ denotes a convolution with dilation l, and $\sigma(\cdot)$ is an activation function (we use GELU for smooth non-linearity). Let d_c be the number of output feature channels for each 3D conv layer. Thus $Q^{(l)}$ is a tensor of shape $N \times L \times L \times d_c$ representing convolved span features for each entity type. We use, for example, three parallel 3D convolutions with dilation $l \in 1, 2, 3$, each producing an output $Q^{(1)}, Q^{(2)}, Q^{(3)}$ respectively. Each of these captures span interactions at a different scale (local, medium, and slightly larger neighborhood). We assign each dilation layer its own set of filters (so they can learn complementary features).

We then concatenate the outputs along the feature channel dimension to aggregate information from all dilation scales (resulting in a combined feature size of $3d_c$ for each span). Finally, we apply a feed-forward network to produce the output scores for boundary labels. Specifically, a small MLP takes as input the concatenated feature vector for each span (for each entity type) and outputs a length-M score vector, one score for each boundary label category:

$$Q = \mathrm{MLP}(Q^{(1)} \oplus Q^{(2)} \oplus Q^{(3)}) \tag{20}$$

Here $\oplus$ denotes concatenation of the feature maps along the channel axis. The result Q has shape $N \times L \times L \times M$. In other words, for each entity type c and span (i, j), $Q[c, i, j, :]$ is a score vector over the boundary labels C, S, E, SE, None as predicted by the CNN module. Notably, this Q is in the same format as the tensor P from the pointer module, but it is derived from an entirely different computational pathway (convolutional instead of attentional). The CNN is trying to predict which boundary label fits each span by looking at the configuration of neighboring spans, thus capturing phenomena like "if span (i, j) is an entity, the spans immediately adjacent to it might have certain characteristic labels" (for example, the span one token larger might be labeled SE, etc.). By using multiple dilation rates, the CNN can also capture nested entity configurations (e.g., a smaller span with C inside a larger span with C will produce a distinctive pattern of S and E around them.

3.2.4 Pointer-CNN fusion and training objective

The two modules above produce complementary predictions: $P[c, i, j, m]$ scores primarily reflect entity-focused evidence (with some boundary conditioning), while $Q[c, i, j, m]$ scores reflect boundary pattern evidence (with entity-type conditioning). We combine these outputs to form the final prediction. In our design, we fuse the pointer and CNN outputs by an element-wise addition of their score tensors (other fusion strategies like concatenation followed by a linear layer are also possible). Let O denote the fused output tensor. We compute:

$$O = P + Q \tag{21}$$

where P and Q are of the same shape $N \times L \times L \times M$, and the addition is performed for each corresponding element (c, i, j, m). Thus, $O[c, i, j, m]$ is the combined score

for span (i, j) with entity type c having boundary label m, taking into account both the global pointer's view and the CNN's view. (If one prefers, O can be seen as the logit output of the joint model before applying a softmax over the M boundary categories.) In our implementation, this fusion is done at inference time as well as backpropagated during training, effectively allowing the pointer and CNN components to learn to complement each other's outputs.

While simple, element-wise addition was chosen for its efficiency and ability to directly integrate complementary span-level and boundary-level information without introducing extra parameters or model complexity. We also considered attention-based fusion mechanisms (e.g., weighted averaging with learned attention scores), but initial experiments showed marginal improvements (<0.5 F1) at the cost of slower training and inference. As such, we opted to retain the element-wise strategy. Detailed comparisons are included in the supplementary material.

With O in hand, the model's goal is to match the ground-truth joint span-boundary matrix for the sentence. We treat this as a multi-class classification for each potential span. For each cell (c, i, j) (each potential span for each entity type), we have a predicted distribution over M labels (the softmax of $O[c, i, j, :]$) and a one-hot ground truth vector indicating which boundary label is correct for that span. We train the network by minimizing the cross-entropy between the predicted and true distributions over all such cells. The loss function $\mathcal{L}$ is given by:

$$\mathcal{L} = -\frac{1}{NL^2} \sum_{c=1}^{N} \sum_{i=1}^{L} \sum_{j=1}^{L} \mathbf{y}_{c,i,j}^{\top} \log \hat{\mathbf{y}}_{c,i,j} \tag{22}$$

where $\mathbf{y}c, i, j \in 0, 1^M$ is the one-hot ground truth label vector for the span at entity type c (it has a 1 in the position of the correct boundary label and 0 elsewhere), and $\hat{\mathbf{y}}c, i, j$ is the model's predicted probability vector (softmax of $O[c, i, j, :]$) for that span. The double summation over i, j covers all L^2 span positions (including invalid ones where $i > j$, which are simply always labeled None in the ground truth), and the outer sum averages over the N entity type layers. This loss effectively sums the cross-entropy for every possible span in the joint matrix. By normalizing over NL^2, we ensure the loss scale is invariant to sentence length or number of entity types. Minimizing $\mathcal{L}$ trains the model to assign high scores (and hence high predicted probability) to the correct boundary label for each span in each entity-type layer, i.e. to accurately reconstruct the joint span-boundary label matrix O to match the annotated matrix. We optimize this loss with standard methods (Adam optimizer, etc.), training all components (BERT, BiLSTM, pointer, CNN) jointly. The result is a learned model that outputs a filled joint matrix for any new sentence.

3.2.5 Span decoding and entity extraction

At inference time, the model produces the fused score tensor $O \in \mathbb{R}^{N \times L \times L \times M}$ for a given sentence. We apply an arg max over the boundary label dimension to obtain the predicted joint span-boundary matrix. That is, for each entity type c and span (i, j), we determine the highest-scoring boundary label:

$$\hat{b}_{c,i,j} = \arg \max_{m \in \{1,\ldots,M\}} O[c, i, j, m] \tag{23}$$

Let $F \in C, S, E, SE, \text{None}^{N \times L \times L}$ denote the resulting predicted label matrix, where $F[c, i, j] = \hat{b}c, i, j$ is the label with index $\hat{b}c, i, j$ (e.g., 1 = C, 2 = S, etc.). Thus, F has the same format as our ground-truth tensor from now filled with the model's predicted labels. Each cell $F[c, i, j]$ indicates that the model believes the span (i, j) is of entity type c with a certain boundary status. For example, if $F[, \text{Person}, , 5, 8] = S$, it means the model predicts that the span from position 5 to 8 in the sentence (inclusive) is adjacent to a Person entity (specifically covering the start boundary of a Person entity). If $F[c, i, j] = \text{None}$, it means span (i, j) is not considered related to any entity of type c.

The final step is to extract actual entity mentions from this matrix F. We do so by scanning for spans labeled C, since these indicate candidate entity spans. Let us define an entity candidate as any span (i, j) for which $F[c, i, j] = C$ for some entity type c. Each such span is a predicted entity of type c, but we will validate it using its boundary predictions. For a predicted span (i, j) of type c (with $F[c, i, j] = C$), we look at its neighboring cells in F:

- The span's left neighbor: $(i - 1, j)$ in the same c layer, which would ideally have label S (start boundary).
- The span's right neighbor: $(i, j + 1)$ in the same layer, which ideally has label E (end boundary).
- The span's surround: $(i - 1, j + 1)$ in the same layer, which ideally has SE (both boundaries).

Each of these neighboring spans provides evidence that the span (i, j) is correctly delimited by boundaries. We count how many of these boundary conditions are satisfied by F. Let $\kappa_{c,i,j}$ be the number of boundary neighbors of (i, j) (in layer c) that are labeled correctly (i.e., $(i-1, j)$ is S or SE in F, $(i, j+1)$ is E or SE, and $(i-1, j+1)$ is SE if applicable). We then apply a threshold τ to decide if the entity is valid. If $\kappa_{c,i,j} \geq \tau$, we consider the span (i, j) a confirmed entity of type c and include it in the final entity list; if $\kappa_{c,i,j} < \tau$, we discard it, reasoning that the model did not sufficiently identify the supporting boundaries for that span. In practice, τ can be set to, for example, 1 or 2. A higher τ means we require more boundary evidence to accept an entity. Setting

$\tau = 0$ would accept every span labeled C without checking boundaries, whereas $\tau = 2$ would require both a correct S and E (or an SE) to be present. This thresholding acts as a precision-enhancing filter to reduce false positives. In our experiments, we found $\tau = 1$ (at least one boundary correctly predicted) to be a reasonable trade-off, but τ can be tuned on a validation set.

Through this decoding procedure, we obtain the set of predicted named entities in the sentence along with their types. The incorporation of boundary label predictions in the decision process ensures that the model not only proposes entity spans but also double-checks that each proposed entity is delimited by consistent boundaries. This leads to higher precision in identifying exact entity spans, especially in cases of nested or adjacent entities where boundary clarity is crucial. In summary, our approach produces a rich output structure that, after decoding, yields the final recognized entities, each supported by the model's learned span-boundary reasoning.

4 Experiments

In this section, we describe the datasets and experimental setup, then present our results compared to baseline models, followed by ablation studies and analysis.

4.1 Datasets and evaluation

We evaluate our approach on six widely-used NER datasets (Table 1), covering both flat and nested entity recognition in multiple languages:

- **CoNLL-2003**: This English dataset consists of newswire articles and focuses on four entity types: Person, Organization, Location, and Miscellaneous. It is a standard benchmark for flat NER.
- **MSRA**: A large Chinese NER dataset from MSRA (Microsoft Research Asia), containing news articles. It has three entity types (Person, Organization, Location) and is a classic benchmark for Chinese flat NER.
- **Weibo**: A Chinese dataset collected from Sina Weibo (a social media platform) annotated for NER. It contains four entity categories, similar to CoNLL (PER, ORG, LOC, GPE). Social media text is noisy, making this dataset challenging.
- **Resume**: A Chinese dataset of job resumes, with each token labeled for entities such as name, education, organization, etc. This dataset has eight entity types and features domain-specific, densely labeled text with relatively consistent formatting.
- **GENIA**: A biomedical domain dataset (English) consisting of 1,999 MEDLINE abstracts. It features nested entities such as proteins, DNA, RNA, cell types, and cell lines (5 entity types in total). Many entities are nested within larger ones (e.g., a protein name inside a DNA phrase), making it a prototypical nested NER benchmark.
- **ACE2005**: The ACE 2005 multilingual corpus includes texts from newswire, broadcasts, weblogs, etc., annotated with entities, relations, and events. We use the English portion for nested NER evaluation. It contains 7 entity types (such as Person, Organization, GPE (geopolitical entity), Location, Facility, Weapon, Vehicle). Nested entities occur when, e.g., a PERSON name is part of an ORGANIZATION entity.

For all datasets, we adopt the conventional micro-averaged Precision (P), Recall (R), and F1-score as evaluation metrics. These are computed by aggregating true positives, false positives, and false negatives across all entity types before calculating the final scores. An entity prediction is considered correct only if both its span boundaries and type exactly match a reference (gold) entity; partial matches are counted as errors. For nested datasets, we follow an exact-match evaluation strategy where each gold entity is evaluated independently, regardless of whether it is nested within another entity. This ensures that overlapping entities are properly assessed. Model thresholds (described later) are tuned on development sets or hold-out validation subsets; for datasets without predefined dev sets (e.g., MSRA, Weibo, Resume), a small portion of training data is reserved for this purpose. All reported results are from the test sets.

Table 1 Statistics of the NER Datasets

Datesets	Type	S-Train	S-Val	S-Test	S-Avg	E-Train	E-Val	E-Test	E-Avg
CoNLL-2003	4	17291	/	3453	14.39	29441	/	5648	1.43
MSRA	3	46471	/	4376	45.54	74703	/	6181	3.24
Weibo	4	1350	270	270	54.57	1855	379	409	2.62
Resume	8	3819	463	477	31.18	13438	1497	1630	5.87
GENIA	5	16692	/	1854	25.42	50509	/	5506	1.95
ACE 2005	7	7178	960	1051	20.58	25300	3321	3099	2.42

4.2 Implementation details

Our BGNER model is implemented in PyTorch. For the encoder, we use the pretrained BERT language model to initialize token representations. Specifically, for English datasets (CoNLL-2003, ACE2005, GENIA) we use BERT-large-cased embeddings, and for the biomedical GENIA we initialize from BioBERT v1.1 (which is BERT-base trained on biomedical text) for better domain adaptation. For Chinese datasets (MSRA, Weibo, Resume), we use BERT-base Chinese. In all cases, the BERT output for each token (after WordPiece subword pooling) has dimensionality 768 (1024 for BERT-large). We append special boundary category tokens ($M = 5$ for labels C, S, E, SE, None) and entity type tokens (N equal to number of entity types in the dataset) to the input sequence as described in Section 2. The BERT encoder outputs contextualized embeddings for all these tokens. We then apply a single-layer BiLSTM on top of BERT to further refine contextual representations. The BiLSTM has a hidden size of 256 in each direction (so $d = 512$ for concatenated output $\mathbf{h}_k$). We found that adding this BiLSTM yields a small improvement (+0.2 F1) by capturing token interactions not emphasized by BERT's subword modeling (this is consistent with observations in previous works combining BERT with BiLSTM). After the encoder, we obtain three sets of embeddings: H^B for the M boundary tokens, H^Y for the N entity type tokens, and H^T for the L actual token positions. The initial learning rate is set to 5e-5 with a linear decay schedule and a warm-up ratio of 0.1. Models are trained for 10 epochs with early stopping based on development set performance. We use a batch size of 32 and apply gradient clipping with a maximum norm of 1.0. The random seed is fixed to 42 to ensure reproducibility. These settings are consistent across all datasets unless otherwise specified.

For the Boundary-Guided Global Pointer module, we set the dimension of CLN-transformed token embeddings to $d = 512$ (same as BiLSTM output). The CLN uses learned scale and bias parameters (W_γ, W_β) to modulate token representations with each boundary token vector. We thus obtain a tensor $U \in \mathbb{R}^{M \times L \times d}$ of boundary-conditioned token features. We then compute span scores using the efficient GlobalPointer formulation: for each boundary category m and each possible span (i, j), we compute a score $s_{m,\beta}(i, j)$ for span being of entity type β using a bilinear term for span boundaries and a linear term for entity type β. We incorporate RPE to inject relative span length information into the boundary scoring. This results in a score tensor $P[c, i, j, m]$ for every entity type c, span (i, j), and boundary label m. P has shape $N \times L \times L \times M$. The pointer module is thus entity-aware (indexed by c) and boundary-aware (indexed by m) in its scoring.

For the Entity-Guided 3D CNN module, we first construct span representation tensor $C \in \mathbb{R}^{N \times L \times L \times d'}$ as described in Section 3.2.3. We include relative distance (span length) embeddings and region indicators (boundary vs interior) in the span features, by concatenating them and passing through an MLP. We use bucketed span length encoding (with buckets for 1, 2, 3-4, 5-8, >8 tokens length) as the distance feature, and binary indicators for whether token i or j are at span boundaries. The resulting span feature dimension d' is 256. Then for each entity type c, we apply a conditional layer normalization with the corresponding type token embedding h_c^Y to get an entity-specific span grid C_c. We use a 3D CNN over this tensor: specifically, we apply three parallel 3D convolution filters with different dilation rates $l \in 1, 2, 3$ across the span dimensions (height i, width j) while keeping the entity type dimension separate. Each 3D convolution layer has a kernel size of $3 \times 3 \times 1$ ($\text{span}_i \times \text{span}_j \times \text{entity}_c$) and 64 output channels, and uses GELU non-linearity. The dilated convolutions allow us to capture interactions among spans that are near each other or have one span apart (for $l = 2$) or two spans apart ($l = 3$), covering a larger context without many layers. The outputs of the three dilation layers (each of shape $N \times L \times L \times 64$) are concatenated along the channel dimension, yielding a Q tensor of shape $N \times L \times L \times 192$. This Q represents boundary pattern scores learned by the CNN. We then apply an output linear layer (small MLP) on Q to map it to the same shape $N \times L \times L \times M$, producing scores $Q[c, i, j, m]$ for each span's boundary label (for each entity type).

The Pointer and CNN Fusion is done by simple element-wise addition: $O = P + Q$. We found this direct fusion effective and it keeps the interpretation of scores intact (both modules contribute logits for the joint classification). During training, we apply softmax over the M boundary labels for each (c, i, j) and use a categorical cross-entropy loss against the ground truth label (C, S, E, SE or None). We average the loss over all $N \times L \times L$ possible spans (note: spans with $i > j$ are invalid and are always labeled None in truth, which the loss handles gracefully). We optimize with Adam (learning rate $2e-5$ for BERT and $1e-3$ for other parts) and train for up to 50 epochs, using early stopping on dev F1. Gradients are clipped at norm 5. We also apply dropout rate 0.1 to the BiLSTM and to the span feature MLP to prevent overfitting.

At inference time, we obtain the fused score tensor O and derive the predicted label for each span: $\hat{b}_{c,i,j} = \arg\max_m O[c, i, j, m]$. This gives us a predicted entity-boundary label matrix F of size $N \times L \times L$ with entries in C, S, E, SE, None. We then extract entities by scanning for any span (i, j) such that $F[c, i, j] = C$ (predicted center of an entity of type c). For each such span, we verify its boundary labels: we check $F[c, i-1, j]$ (left-neighbor span) for S or SE, $F[c, i, j+1]$ (right-neighbor) for E or SE, and $F[c, i-1, j+1]$ (surrounding span) for SE. We count how

many of these conditions are satisfied. If at least $\tau = 1$ of them holds (i.e., at least one boundary neighbor is correctly predicted), we accept the span as an entity. If not, we discard it as a likely false positive. We chose $\tau = 1$ based on development experiments - this setting yields a good precision/recall trade-off, filtering out many spurious spans while keeping most true entities (a higher threshold $\tau = 2$ would be more strict, requiring both sides' boundaries, but we found it sometimes drops recall without significant precision gain). This decoding step is linear in the number of spans and had negligible impact on runtime, but significantly improved precision on nested datasets by removing inconsistent predictions. For example, if the model predicted a span as C but did not predict any S or E around it, it often turned out to be an incorrect entity - and our rule filters that out. The key hyperparameter and hardware configurations are summarized in Table 2.

4.3 Baselines

We compare BGNER with a broad range of baseline models, including both classic approaches and recent state-of-the-art methods, to demonstrate its competitiveness:

- BiLSTM-CRF (Zhang et al. 2024): A standard BiLSTM with a CRF layer for sequence labeling. We include this to represent traditional sequence tagging performance.
- BERT-Tagger (Ke et al. 2024): A strong contextual baseline that uses a pretrained BERT (or Chinese BERT) and a linear classifier for token tagging.
- Lattice LSTM (Li et al. 2021): A lattice-based BiLSTM model that incorporates lexicon embeddings for Chinese NER. It's a representative of lexicon-enhanced models for Chinese.
- FLAT (Liu et al. 2019): The Flat-Lattice Transformer, which is the Transformer counterpart of lattice LSTM, known to perform very well on Chinese benchmarks by encoding lexicon information in self-attention.
- SoftLexicon (Raffel et al. 2020): The method augments a BERT encoder with lexicon features via a lexicon embedding table. We include this for Chinese datasets.
- Layered (Sutton and McCallum 2006): The layered BiLSTM-CRF approach for nested NER by Ju et al. We evaluate it on ACE2005 and GENIA (nested tasks) to compare to our span-based approach.
- Pyramid (Graves and Schmidhuber 2005): Pyramid model for nested NER, which we include on nested datasets.
- Biaffine (Huang et al. 2015): The span-based biaffine scorer model, representing the first neural span enumeration approach. We use their reported results or reimplementation to get its performance on our datasets.
- W2NER (Jehangir et al. 2023): A recent model that treats NER as word-to-word relation classification. It encodes adjacent word relations and was shown to excel on flat NER and handle nested structures to some extent.
- Locate&Label (Keskar et al. 2019): Two-stage nested NER model with boundary proposal and labeling.
- MECT (Strubell et al. 2020): A Multi-Feature Cross-Transformer for Chinese NER.
- MFT+BERT (Yan et al. 2021): A strong Chinese NER baseline that fuses diverse features into BERT. We found this was one of the top performers on Chinese datasets.
- Triaffine (Xu and OuYang 2023): Span-based model with triaffine attention, which fuses token, boundary, and label information.
- Boundary Smooth (Pakhale 2023): The boundary smoothing regularization method mentioned earlier. We applied their technique on top of a baseline span model to obtain results.
- DiffusionNER (Yadav and Bethard 2019): The diffusion-based NER model, which we compare on both flat and nested benchmarks.
- CNN-NER (Sohrab and Miwa 2018): The 2D CNN on span score matrix . This can be seen as a simplified ver-

Table 2 Training hyperparameters and hardware settings

Setting	Value
Batch size	32
Learning rate (BERT modules)	2e-5
Learning rate (non-BERT modules)	1e-3
Maximum number of training epochs	50
Early stopping criterion	Based on development set F1 score
Dropout rate	0.1 (applied to BiLSTM and span MLP layers)
Optimizer	Adam
Learning rate warm-up	Linear warm-up with 10% of total steps
Gradient clipping	Max norm 1.0 for encoder; 5.0 for pointer and CNN modules
Random seed	42
GPU type	NVIDIA A100

Table 3 Results for the English ffat NER dataset CoNLL2003

	CoNLL-2003		
	P	R	F
BiLSTM-CRF	-	-	91.03
BERT-Tagger	-	-	92.80
Boundary Smooth	92.89	93.21	93.06
W2NER	92.71	93.44	93.07
Biaffine	92.46	95.67	92.55
DiffusionNER	92.99	92.56	92.78
Our	**93.18**	**94.15**	**93.23**

sion of our CNN idea (but without boundary labels or an integrated approach).

- SEGP (Span+Efficient Global Pointer) (Athiwaratkun et al. 2020): An efficient GlobalPointer-based span model that addresses class imbalance (segment enhanced GP). This is a recent variant of GlobalPointer specialized for NER.

4.4 Main results

Flat NER results (English) Table 3 shows the performance on the CoNLL2003 English dataset. As a high-resource flat NER benchmark, CoNLL-2003 tends to yield near-ceiling performance across many models. Our proposed model achieves an F1 score of 93.23, slightly outperforming the previous best model W2NER (93.07 F1) and other strong baselines such as Boundary Smooth (93.06) and BERT-Tagger (92.80). Compared with W2NER, our model shows a +0.16% absolute improvement in F1, reaffirming its strength even in flat NER scenarios. In terms of precision and recall, our model achieves the highest precision (93.18) and highest recall (94.15) among all compared methods, suggesting a well-balanced ability to accurately and comprehensively identify named entities. Particularly, the significant recall gain indicates that our method is more effective at identifying difficult or ambiguous entities that other models tend to miss.

Flat NER results (Chinese) Table 4 reports F1 scores on three Chinese flat NER datasets: MSRA, Weibo, and Resume. Our model achieves the best F1 scores across all three benchmarks: 96.38 on MSRA, 73.01 on Weibo, and 97.05 on Resume. These results respectively outperform the strongest baselines by +0.12, +0.69, and +0.40, reaffirming the effectiveness of our boundary-guided approach. The largest gain appears on the Weibo dataset, which is widely considered the most challenging due to its informal language, high variability, and limited training data. Our model reaches 73.01 F1, significantly ahead of models like Boundary Smooth (71.41) and W2NER (72.32). We attribute this improvement to our boundary span mechanism, which provides additional structural cues to better detect entities in noisy or low-context text. Notably, our model shows a recall improvement of nearly +1.4 points over most baselines, indicating a stronger ability to capture difficult entities with only a minimal trade-off in precision. On MSRA and Resume, which have more regular and formal structure, our model still delivers consistent improvements over prior state-of-the-art methods. While models like W2NER and Boundary Smooth already perform strongly on these datasets, our model matches or slightly exceeds their F1 scores while maintaining high precision-recall balance. For example, on Resume, our model reaches 97.05 F1, outperforming W2NER (96.65) and MFT+BERT (96.01), despite not using external resources such as lexicons or phonetic features. Among baselines, MFT+BERT (Multi-Feature Transformer) performs competitively on MSRA and Resume by leveraging explicit features (e.g., lexicons, pinyin, radical), showing that integrating linguistic priors is helpful. However, our model outperforms it without relying on such external input, demonstrating that the boundary-aware training signal serves as a strong inductive bias - essentially acting like a regularization or soft data augmentation mechanism that improves generalization. In summary, our method sets new state-of-the-art performance on Chinese flat NER across datasets of varying difficulty. This further confirms that our span-boundary matrix and boundary-guided neural model are robust and adaptable across both formal and informal domains, and across different scripts and language structures.

Nested NER results Table 5 presents the performance on two English nested NER datasets: ACE2005 and GENIA. Our model achieves 87.21 F1 on ACE2005 and 82.05 F1 on GENIA, outperforming all compared baselines. These gains, though numerically modest (+0.28 over the best prior on ACE and +0.52 on GENIA), are significant in the context of nested NER, where overall F1 scores tend to saturate in the low-to-mid 80s and further improvements are difficult to obtain. On ACE2005, our model attains the highest F1 score of 87.21, slightly surpassing prior top performers like DiffusionNER (86.93) and CNN-NER (86.82). It also achieves strong balance between precision (86.72) and recall (88.63), indicating that it captures both exact boundaries and diverse entity occurrences reliably. Compared to DiffusionNER, which uses a powerful but complex generative framework, our discriminative, boundary-aware design yields more accurate boundary localization with simpler decoding. On GENIA, our model sets a new best F1 score of 82.05, edging past previous models such as Triaffine (81.53), W2NER (81.39), and CNN-NER (81.40). Notably, our model attains precision and recall that are nearly identical (83.19 / 82.07), showing stable generalization even in the biomedical

Table 4 Results for the Chinese ffat NER datasets MSRA, Weibo and Resume

	MSRA			Weibo			Resume		
	P	R	F	P	R	F	P	R	F
DiffusionNER	95.71	94.11	94.91	-	-	-	-	-	-
Flat	-	-	96.09	-	-	68.55	-	-	95.86
MECT	-	-	96.24	-	-	70.43	-	-	95.98
MFT+BERT	-	-	-	68.33	71.09	69.71	96.24	95.77	96.01
W2NER	96.12	96.08	96.1	70.84	73.87	72.32	96.96	96.35	96.65
Boundary Smooth	96.37	96.15	96.26	68.65	74.40	71.41	96.63	96.69	96.66
Lattice	93.57	92.79	93.18	53.04	62.25	58.79	94.81	94.11	94.46
SoftLexicon	95.75	95.10	95.42	70.94	67.02	70.50	96.08	96.13	96.11
Our	**96.38**	**96.45**	**96.38**	**70.98**	**75.29**	**73.01**	**96.97**	**97.19**	**97.05**

domain, where entities are often long, domain-specific, and deeply nested. GENIA contains frequent cases of short, one-token entities nested inside longer biomedical phrases. These were commonly overlooked by previous span-based models due to their length biases. Our model's boundary labels (S/E) help highlight such entities by explicitly marking short but valid spans, resulting in improved recall without hurting precision. Furthermore, our model consistently outperforms even specialized nested NER architectures like Pyramid and Locate-and-Label, despite using a single-model design without extra modules or postprocessing stages. This highlights the effectiveness of our unified span-boundary matrix formulation. By modeling both span existence and boundary structure jointly, the network naturally infers nested configurations - for instance, an inner entity is marked with a span label "C" and surrounded by corresponding S/E boundaries, even if nested inside a longer span. In conclusion, our model delivers state-of-the-art nested NER performance across both general and biomedical English domains. These results validate our hypothesis that joint boundary-span modeling provides a compact yet expressive inductive bias, which improves both boundary precision and nested structure recovery.

4.5 Ablation study

To understand the contribution of each component in our architecture, we conduct ablation experiments. Table 6 summarizes the results of various ablated versions of our model:

- **Full BGNER**: The complete model as described, using boundary labels, CLN integrations, and the 3D CNN module.
- **w/o Boundary spans**: In this setting, we remove boundary labels from training and prediction - effectively turning off the "boundary span" aspect. Concretely, we train the model to only predict entity vs none for each span (similar to a standard span classification). The global pointer then only scores spans for entity types without any S/E/SE labels, and the 3D CNN (if used) operates on a single-channel span matrix. This ablation measures the value of our joint entity-boundary labeling.
- **w/o CLN**: Here we remove all Conditional Layer Normalization usage. Instead of fusing boundary token embeddings via CLN in the pointer module, we might simply concatenate or add them (or not use them at all). In our implementation of this ablation, we excluded the boundary token conditioning on the token representations (effectively using the token embeddings H^T directly for span scoring) and also excluded the entity token conditioning in the CNN (just using the span feature matrix S without type-specific normalization).
- **w/o 3D CNN**: In this ablation, we remove the 3D CNN module entirely, using only the boundary-guided pointer to make predictions. This is akin to a purely GlobalPointer-based span model that uses boundary token conditioning (since we left the CLN in pointer on for a fair comparison).

Table 5 Results for the English nested NER datasets ACE 2005 and GENIA

	ACE 2005			GENIA		
	P	R	F1	P	R	F1
Layered	74.20	70.30	72.20	78.50	71.30	74.70
SEGP	86.00	84.29	84.72	81.03	78.64	80.40
Pyramid	83.95	85.39	84.66	79.45	78.94	79.19
Biaffine	85.20	85.60	85.40	78.20	78.20	78.20
Locate and Label	86.09	87.27	86.67	80.19	80.89	80.54
W2NER	85.03	88.62	86.79	83.10	79.76	81.39
Triaffine	86.70	86.94	86.82	80.24	82.06	81.53
CNN-NER	86.39	87.24	86.82	83.18	79.70	81.40
DiffusionNER	86.15	87.72	86.93	82.10	80.97	81.53
Ours	**86.72**	**88.63**	**87.21**	**83.19**	**82.07**	**82.05**

Table 6 Results for the ablation study

	CoNLL-2003	MSRA	Weibo	Resume	ACE 2005	GENIA
BGNER	**93.23**	**96.38**	**73.01**	**97.05**	**87.21**	**82.05**
w/o Boundary spans	92.87	95.15	72.25	96.75	86.52	81.76
w/o CLN	92.61	95.01	72.15	96.25	86.15	81.25
w/o 3D CNN	92.54	94.98	72.01	96.12	86.11	81.01

Table 6 presents an ablation study evaluating three key components of our BGNER model: boundary span supervision, Conditional Layer Normalization (CLN), and the 3D CNN module. Removing boundary supervision consistently reduces F1 scores, particularly on nested datasets like GENIA (82.05 → 81.76) and ACE2005 (87.21 → 86.52), confirming its importance for handling nested structures. Flat datasets (e.g., CoNLL, Resume) also show moderate declines, indicating its general utility for span precision. Excluding CLN leads to further performance drops, demonstrating its role in refining predictions via boundary-aware conditioning. The 3D CNN has limited impact on flat datasets but noticeably affects nested ones, supporting its design for modeling span interactions. Overall, each component contributes meaningfully, and even without any single one, the model remains competitive (e.g., 92.87 on CoNLL without boundary spans), highlighting the strength of our pointer+boundary framework.

4.6 Error analysis

While BGNER advances the state of the art, there are still some challenges and errors to analyze. For flat NER, most errors come from boundary ambiguities - ironically, the very issue we aim to solve. For instance, in Chinese Resume NER, our model occasionally missed the last character of an entity if that character is itself an entity in another context. Boundary modeling helped reduce such errors but not eliminate them entirely. In nested NER, we examined GENIA outputs and found that certain entity types are harder for our model: e.g., DNA vs RNA vs protein mentions can be nested and lexically similar, and if the model predicts the wrong type (it might label a span as Protein when it was a DNA), the boundary labels around it can all be consistent but the type is wrong. This type confusion is something our boundary labels do not directly address, as they help with spans more than with type semantics. Another common error pattern in GENIA was missing very short entities (single-token entities like "T-cell"). If the token by itself doesn't give away its entity nature, the model might not assign C to that 1-length span. We do use boundary neighbors (S/E) to try to catch these, but if both sides are also ambiguous, the model can still miss it. We think incorporating character-level features or better entity-type representations (maybe through a biomedical dictionary) could help here, but that is outside our current scope.

On ACE2005, a notable difficulty was adjacent entities: sometimes two entities of the same type occur back-to-back with no space (common in Chinese, or in English like "Washington, D.C." where "Washington" and "D.C." are separate Location entities). Our model has to label one span as C and possibly the span covering both as None (or one of them as boundary of the other). We found a few cases where BGNER incorrectly merged them or split one entity into two. This is a complex scenario because it blurs the line between what is a boundary vs what is an entity start.

In summary, our analysis indicates that while boundary information greatly helps in demarcating spans, there is room for improvement in entity type disambiguation and in handling edge cases like contiguous entities and extremely short or long entities. Future work could enhance the model's type representations (perhaps using knowledge bases or gazetteers for certain domains) and explore more advanced decoding strategies (e.g., learning the optimal τ or a learned module to validate spans instead of a fixed rule). Nonetheless, the overall reduction in span boundary errors with BGNER is evident: in our manual inspection, we saw many cases where a baseline span model would propose an entity that was slightly too long or too short, whereas BGNER got the boundaries exactly right, thanks to leveraging the boundary span cues. This demonstrates the effectiveness of the boundary-enhanced span representation in practice.

4.7 Scalability analysis on long sequences

To assess the scalability and practical viability of our framework, we conducted a preliminary evaluation of inference time on input sequences of varying lengths. Specifically, we compared our model with a representative span-based baseline (BERT-base with span classification head) on sequences of 512, 1024, and 2048 tokens.

Table 7 Inference time (ms/sample) vs. sequence length.

Sequence Length	512 tokens	1024 tokens	2048 tokens
Span-based Model	37.5 ms	82.1 ms	165.4 ms
Our Model	33.2 ms	56.8 ms	91.7 ms

As shown in Table 7, our method demonstrates significantly improved inference efficiency as sequence length increases. While both models perform comparably on shorter inputs, our framework achieves notably lower latency on longer sequences due to its optimized representation mechanism and reduced reliance on exhaustive span enumeration.

These results suggest that our approach scales more favorably with increasing input length and offers practical advantages for deployment in real-world applications where long-context reasoning is required.

5 Conclusion

We presented BGNER, a boundary span-based approach to NER that integrates boundary context into span-level entity recognition. Our method introduces an entity-boundary span matrix which enriches the traditional span representation by labeling each span not just as entity or non-entity, but also indicating whether it touches an entity's boundaries. We designed a neural network with a Category-Enhanced encoder, a Boundary-Guided Global Pointer, and an Entity-Guided 3D CNN to effectively learn from this representation. Through comprehensive experiments on six benchmarks (covering both flat and nested NER in English and Chinese), we demonstrated that BGNER achieves state-of-the-art performance, validating the benefit of boundary-aware modeling. In particular, our approach excels in complex scenarios with nested or adjacent entities, where it outperforms prior models by a clear margin. Our ablation studies confirmed that each component of BGNER - the boundary span formulation, the conditional layer normalization fusion, and the 3D CNN - contributes to its success. The proposed decoding strategy, which uses predicted boundary spans to guide entity decisions, was shown to improve precision without harming recall. This highlights an interpretable advantage of our approach: the model doesn't just output entities, it also outputs why it thinks those spans are entities in terms of boundary evidence. In future work, we plan to explore further generalizations of boundary-informed NER. One promising direction is to extend the idea to discontinuous entity recognition, where entities may have internal gaps; our span-boundary matrix might be extended with pointers linking spans that form a discontinuous entity. Furthermore, although the current formulation supports only contiguous spans within single sentences, it could be theoretically extended to handle cross-sentence entities. This would involve modeling span continuity across sentence boundaries, potentially by expanding the context window and enabling span-linkage across sentence-level segments. These directions present exciting challenges for generalizing span-boundary frameworks. Another direction is to incorporate external knowledge (e.g., gazetteers or ontology information) by encoding it in the special category tokens to further improve entity type prediction. Additionally, while our current model focuses on character-based span boundaries, incorporating semantic boundaries (like sentence or clause boundaries) could be beneficial in long texts. In conclusion, this work shows that modeling what lies just outside an entity is as important as modeling the entity itself. By formally incorporating boundary spans into the NER model's design, we can substantially enhance the model's ability to find and delineate entities. We hope our approach encourages further research on multi-dimensional representations for structured prediction, and we believe BGNER can be a strong foundation for advanced NER systems in the future.

Author Contributions Yu He: Methodology, Writing–original draft, Project administration, Data Curation, Software, Formal analysis, Resources.
Li Sun: Conceptualization, Data curation, Methodology, Software, Investigation.
Qianyu Yue: Methodology, Funding acquisition, Validation, Writing–review & editing.
Haitao Wu: Supervision, Methodology, Resources, Funding acquisition, Writing–review & editing.
Xiaojuan Wang: Project administration, Funding acquisition.

Data Availability The datasets used in this manuscript are publicly available datasets. Detailed information about these datasets is provided in Section 4.1 *Dataset* of this manuscript.

Declarations

Conflicts of Interest The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

Alqahtani FF, Mohsan MM, Alshamrani K, Zeb J, Alhamami S, Alqarni D (2024) Cnx-b2: a novel cnn-transformer approach for chest x-ray medical report generation. IEEE Access 12:26626–26635
Ashok D, Lipton ZC (2023) Promptner: prompting for named entity recognition. arXiv:2305.15444

Asif U, Bennamoun M, Sohel FA (2017) A multi-modal, discriminative and spatially invariant cnn for rgb-d object labeling. IEEE Trans Pattern Anal Mach Intell 40(9):2051–2065

Athiwaratkun B, Santos CNd, Krone J, Xiang B (2020) Augmented natural language for generative sequence labeling. arXiv:2009.13272

Chaturvedi R, Baghershahi P, Medya S, Di Eugenio B (2025) Temporal relation extraction in clinical texts: a span-based graph transformer approach. arXiv:2503.18085

Chen X, Zhang W, Pan S (2024) Clesr: context-based label knowledge enhanced span recognition for named entity recognition. Int J Comput Intell Syst 17(1):190

Chi Z, Huang S, Dong L, Ma S, Zheng B, Singhal S, Bajaj P, Song X, Mao X-L, Huang H et al (2021) Xlm-e: cross-lingual language model pre-training via electra. arXiv:2106.16138

Chiu JP, Nichols E (2016) Named entity recognition with bidirectional lstm-cnns. Trans Assoc Comput Linguist 4:357–370

Collobert R, Weston J, Bottou L, Karlen M, Kavukcuoglu K, Kuksa P (2011) Natural language processing (almost) from scratch

Conneau A, Khandelwal K, Goyal N, Chaudhary V, Wenzek G, Guzmán F, Grave E, Ott M, Zettlemoyer L, Stoyanov V (2019) Unsupervised cross-lingual representation learning at scale. arXiv:1911.02116

Dai Z, Yang Z, Yang Y, Carbonell J, Le QV, Salakhutdinov R (2019) Transformer-xl: Attentive language models beyond a fixed-length context. arXiv:1901.02860

Das BC, Amini MH, Wu Y (2025) Security and privacy challenges of large language models: a survey. ACM Comput Surv 57(6):1–39

Didolkar A, Zadaianchuk A, Awal R, Seitzer M, Gavves E, Agrawal A (2025) Ctrl-o: language-controllable object-centric visual representation learning. arXiv:2503.21747

Du M, Zhao Y, Jin G, Ren Y, Cui R, Yin F (2024) Span-based joint entity and relation extraction method. In: 2024 4th Asia-Pacific Conference on Communications Technology and Computer Science (ACCTCS), pp 123–127. IEEE

Fei H, Ji D, Li B, Liu Y, Ren Y, Li F (2021) Rethinking boundaries: end-to-end recognition of discontinuous mentions with pointer networks. In: Proceedings of the AAAI conference on artificial intelligence 35:12785–12793

Graves A, Schmidhuber J (2005) Framewise phoneme classification with bidirectional lstm networks. In: Proceedings. 2005 IEEE international joint conference on neural networks, 2005, vol 4, pp 2047–2052. IEEE

Guo Q, Guo Y (2022) Lexicon enhanced Chinese named entity recognition with pointer network. Neural Comput Appl 34(17):14535–14555

Ha S, Grubert E (2023) Hybridizing qualitative coding with natural language processing and deep learning to assess public comments: a case study of the clean power plan. Energy Res Soc Sci 98:103016

Holtzman A, Buys J, Du L, Forbes M, Choi Y (2019) The curious case of neural text degeneration. arXiv:1904.09751

Hong S-Y, Na S-H, Shin J-H, Kim Y-K (2018) Koelmo: deep contextualized word representations for Korean. In: Annual conference on human and language technology, pp 296–298. Human and Language Technology

Huang R, Chen Y, Huang R (2025) A levitated controlled attention for named entity recognition. Cogn Comput 17(1):1

Huang Y, Chen Y, Li Z (2023) Applications of large scale foundation models for autonomous driving. arXiv:2311.12144

Huang Z, Xu W, Yu K (2015) Bidirectional lstm-crf models for sequence tagging. arXiv:1508.01991

Hu M, Zhang Z, Zhao S, Huang M, Wu B (2023) Uncertainty in natural language processing: sources, quantification, and applications. arXiv:2306.04459

Jehangir B, Radhakrishnan S, Agarwal R (2023) A survey on named entity recognition-datasets, tools, and methodologies. Nat Lang Process J 3:100017

Jia H, Huang J, Zhao K, Mao Y, Zhou H, Ren L, Jia Y, Xu W (2024) Based and attention-enhanced grid-tagging network for mention recognition. Electronics 13(2):261

Jiang L (2024) Improving named entity recognition with multi-feature word-to-word relationship classification. In: 2024 International Joint Conference on Neural Networks (IJCNN), pp 1–8. IEEE

Johnson A, Karanasou P, Gaspers J, Klakow D (2019) Cross-lingual transfer learning for Japanese named entity recognition. In: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, vol 2 (Industry Papers), pp 182–189

Ke X, Wu X, Ou Z, Li B (2024) Chinese named entity recognition method based on multi-feature fusion and biaffine. Complex Intell Syst 10(5):6305–6318

Keskar NS, McCann B, Varshney LR, Xiong C, Socher R (2019) Ctrl: a conditional transformer language model for controllable generation. arXiv:1909.05858

Lafferty J, McCallum A, Pereira F et al (2001) Conditional random fields: probabilistic models for segmenting and labeling sequence data. In: Icml, vol 1, p 3. Williamstown, MA

Lample G, Ballesteros M, Subramanian S, Kawakami K, Dyer C (2016) Neural architectures for named entity recognition. arXiv:1603.01360

Lewis M, Liu Y, Goyal N, Ghazvininejad M, Mohamed A, Levy O, Stoyanov V, Zettlemoyer L (2019) Bart: denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. arXiv:1910.13461

Li F, Wang Z, Hui SC, Liao L, Zhu X, Huang H (2021) A segment enhanced span-based model for nested named entity recognition. Neurocomputing 465:26–37

Liu C, Fan H, Liu J (2021) Span-based nested named entity recognition with pretrained language model. In: Database systems for advanced applications: 26th International conference, DASFAA 2021, Taipei, Taiwan, April 11–14, 2021, Proceedings, Part II 26, pp 620–628. Springer

Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V (2019) Roberta: a robustly optimized Bert pretraining approach. arXiv:1907.11692

Ma X, Hovy E (2016) End-to-end sequence labeling via bi-directional lstm-cnns-crf. arXiv:1603.01354

Mo Y, Yang J, Liu J, Wang Q, Chen R, Wang J, Li Z (2024) Mcl-ner: cross-lingual named entity recognition via multi-view contrastive learning. In: Proceedings of the AAAI conference on artificial intelligence, vol 38, pp 18789–18797

Pakhale K (2023) Comprehensive overview of named entity recognition: models, domain-specific applications and challenges. arXiv:2309.14084

Parsing C (2009) Speech and language processing. Power Point Slides

Qiu Q, Xie Z, Wu L, Tao L (2019) Gner: a generative model for geological named entity recognition without labeled data using deep learning. Earth Space Sci 6(6):931–946

Rabiner LR (1989) A tutorial on hidden Markov models and selected applications in speech recognition. Proceedings of the IEEE, vol 77, no 2, pp 257–286

Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu PJ (2020) Exploring the limits of transfer learning with a unified text-to-text transformer. J Mach Learn Res 21(140):1–67

Roy A (2021) Recent trends in named entity recognition (ner). arXiv:2101.11420

Sarzynska-Wawer J, Wawer A, Pawlak A, Szymanowska J, Stefaniak I, Jarkiewicz M, Okruszek L (2021) Detecting formal thought disorder by deep contextualized word representations. Psych Res 304:114135

Shen Y, Ma X, Tan Z, Zhang S, Wang W, Lu W (2021) Locate and label: a two-stage identifier for nested named entity recognition. arXiv:2105.06804

Sohrab MG, Miwa M (2018) Deep exhaustive model for nested named entity recognition. In: Proceedings of the 2018 conference on empirical methods in natural language processing, pp 2843–2849

Song J, Wang X, Zhang H, Li B, Wang T (2024) A boundary feature enhanced span-based nested named entity recognition method. In: Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) joint international conference on web and big data, pp 3–17. Springer

Strubell E, Ganesh A, McCallum A (2020) Energy and policy considerations for modern deep learning research. In: Proceedings of the AAAI conference on artificial intelligence, vol 34, pp 13693–13696

Su S, Qu J, Cao Y, Li R, Wang G (2022) Adversarial training lattice lstm for named entity recognition of rail fault texts. IEEE Trans Intell Transp Syst 23(11):21201–21215

Su J, Murtadha A, Pan S, Hou J, Sun J, Huang W, Wen B, Liu Y (2022) Global pointer: novel efficient span-based approach for named entity recognition. arXiv:2208.03054

Sun C, Yang Z, Luo L, Wang L, Zhang Y, Lin H, Wang J (2019) A deep learning approach with deep contextualized word representations for chemical-protein interaction extraction from biomedical literature. IEEE Access 7:151034–151046

Sutton C, McCallum A (2006) An introduction to conditional random fields for relational learning. Introduction to statistical relational learning, vol 2, pp 93–128

Thomas A, Sangeetha S (2020) Deep learning architectures for named entity recognition: a survey. In: Advanced computing and intelligent engineering: Proceedings of ICACIE 2018, vol 1, pp 215–225. Springer

Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, Kaiser Ł, Polosukhin I (2017) Attention is all you need. Advances in neural information processing systems, vol 30

Wang Y, Tong H, Zhu Z, Li Y (2022) Nested named entity recognition: a survey. ACM Trans Knowl Discov Data (TKDD) 16(6):1–29

Wu Y, Jiang M, Lei J, Xu H (2015) Named entity recognition in Chinese clinical text using deep neural network. In: MEDINFO 2015: eHealth-enabled health, pp 624–628. IOS Press

Xu W, OuYang J (2023) A multi-task instruction with chain of thought prompting generative framework for few-shot named entity recognition. In: International conference on artificial neural networks, pp 1–15. Springer

Yadav V, Bethard S (2019) A survey on recent advances in named entity recognition from deep learning models. arXiv:1910.11470

Yan H, Gui T, Dai J, Guo Q, Zhang Z, Qiu X (2021) A unified generative framework for various ner subtasks. arXiv:2106.01223

Yang J, Zhang T, Tsai C-Y, Lu Y, Yao L (2024) Evolution and emerging trends of named entity recognition: bibliometric analysis from 2000 to 2023. Heliyon

Yu J, Bohnet B, Poesio M (2020) Named entity recognition as dependency parsing. arXiv:2005.07150

Zaratiana U, Tomeh N, Holat P, Charnois T (2022) Named entity recognition as structured span prediction. In: Proceedings of the workshop on Unimodal and Multimodal Induction of Linguistic Structures (UM-IoS), pp 1–10

Zhang Y, Wang Y (2023) A query-parallel machine reading comprehension framework for low-resource ner. Findings of the Association for Computational Linguistics: EMNLP 2023:2052–2065

Zhang X, Chen G, Cui S, Sheng J, Liu T, Xu H (2024) Exogenous and endogenous data augmentation for low-resource complex named entity recognition. In: Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval, pp 630–640

Zhang M, Zhang Y, Fu G (2017) End-to-end neural relation extraction with global optimization. In: Proceedings of the 2017 conference on empirical methods in natural language processing, pp 1730–1740

Zheng C, Cai Y, Xu J, Leung H, Xu G (2019) A boundary-aware neural model for nested named entity recognition. In: Proceedings of the 2019 conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). Association for Computational Linguistics

Zhu E, Li J (2022) Boundary smoothing for named entity recognition. arXiv:2204.12031